

RAPORT TECHNICZNY Z PROJEKTU NR 5

Autorzy: Natalia Świderek, Szymon Kucharski

Zadanie zrealizowane zostało w oparciu o klasę *Algorithm*, w której znajdują się parametry problemu optymalizacyjnego (macierz Q , wektor p , punkt startowy x_0). Do ewaluacji funkcji zaimplementowana została funkcja $f(x)$ oparta na konstrukcji match-case, co pozwala na dynamiczne przełączanie się między badaną funkcją kwadratową, a funkcją nieliniową bez konieczności modyfikacji wewnętrznej logiki solverów.

Algorytmy Heavy Ball oraz Nesterova opisuje się równaniami drugiego rzędu, które uwzględnia „przyspieszenie” prędkości przemieszczania się punktu w stronę minimum. Ponieważ w kodzie użyta została funkcja *scipy.integrate.solve_ivp*, która wymaga układu pierwszego rzędu, konieczne było zredukowanie rzędu równań poprzez złączenie pozycji x oraz prędkości v w jeden wektor y za pomocą funkcji *np.concatenate*.

Wersje dyskretne algorytmów zaimplementowano jako pętle iteracyjne z dodatkowym warunkiem wczesnego stopu – obliczenia kończą się, gdy aktualna wartość funkcji zbliży się do minimum z określoną dokładnością *tol*. Kolejnym kluczowym aspektem jest dobór parametru alfa. Jeżeli parametr ten nie zostanie podany przez użytkownika, wówczas algorytm sam dopasuje krok. W przypadku funkcji kwadratowej algorytm oblicza optymalny krok na podstawie krzywizny korzystając wartości macierzy Q . Dla funkcji nieliniowej algorytm wywołuje metodę *armijo(x, grad_f)*.

Użycie metody *.copy()* przy dodawaniu kolejnych punktów do listy historii (*xs*), co zapobiega nadpisywaniu referencji w pamięci, co gwarantuje, że trajektorie algorytmów na wykresach zostaną wyrysowane poprawnie, a nie jako zbiór identycznych punktów końcowych.

Dyskretna wersja metody Nesterova wyróżnia się złożonym mechanizmem zarządzania pędem. W każdej iteracji parametr pędu t rośnie zgodnie ze wzorem:

$$t_{new} = \frac{1 + \sqrt{1 + 4t^2}}{2}$$

W teorii powinno to przyspieszać zbieżność. W trybie adaptacyjnym *restart='adaptive'* algorytm w każdym kroku sprawdza, czy wartość funkcji nie zaczęła niespodziewanie rosnąć, tzn. czy zachodzi warunek $f_{new} > f_{old}$. Taka sytuacja oznacza przeskoczenie minimum funkcji i dalsze poszukiwanie. W odpowiedzi algorytm całkowicie resetuje parametr t do wartości początkowej $t = 1.0$, a następnie cofa układ do pozycji x_{old} i stamtąd wznawia poszukiwania.

Ostatnim elementem klasy są metody rysujące, między innymi funkcje *plot_trajectory* lub *plot_function_gap*. Mają one za zadanie stworzyć siatkę punktów za pomocą *np.meshgrid* i narysować na jej podstawie warstwice funkcji przy użyciu *ax.contour*. Na tak przygotowane tło nakładana jest ścieżka, po której poruszał się algorytm, dzięki czemu od razu widzimy historię jego kroków.

Z kolei na wykresach błędów, które pokazują jak daleko punkt znajduje się od szukanego minimum, zastosowano skalę logarymiczną, co znacząco ułatwia odczytanie, jak szybko różne wersje algorytmów (ciągłe i dyskretne) docierają do celu. Taki zabieg pozwala też fajnie sprawdzić, na przykład w metodzie *plot_alpha_sensitivity* jak zmiana parametrów kroku wpływa na to, czy nasz algorytm działa stabilnie i się nie gubi.